



Guia 4: Forward y Backpropagation

1. Backpropagation: Elección múltiple

El algoritmo de retropropagación (backpropagation) puede ser definido como:

- Un algoritmo de optimización que sólo sirve para entrenar redes neuronales. ☐
- Un método de regularización para redes neuronales. ☐
- Un algoritmo para el cálculo del gradiente de una función. ☐
- Un algoritmo para el cálculo del mínimo de una función. ☐
- Un algoritmo para el cálculo del máximo de una función. ☐
- Un algoritmo de optimización que puede ser utilizado para encontrar extremos absolutos (mínimos y máximos) en cualquier función. ☐

2. PyTorch: Elección múltiple

a. PyTorch es una biblioteca de cómputo con tensores para Python que permite diferenciar funciones por medio de:

- Diferenciación automática. ☐
- Diferenciación simbólica. ☐
- Diferenciación numérica. ☐

b. PyTorch es una biblioteca de cómputo con tensores para Python que:

- Implementa diferenciación simbólica en lugar de diferenciación automática. ☐
- Permite paralelizar operaciones en múltiples procesadores de CPU y GPU. ☐
- Permite aplicar aprendizaje por mini batch mediante métodos que cargan los datos en memoria como grupos o lotes. ☐
- Sólo se puede utilizar para desarrollar modelos de visión computacional. ☐

3. Desarrollo

Dadas las funciones:

$$f = -(max(z, x2) + 2(w * y))$$

evaluada en el punto

$$x = 2; z = 1; w = 3; y = 4.$$

$$g = -3((x * 4y) + max(w, z))$$

evaluada en el punto

$$x = 5; y = 2; w = 3; z = 4.$$

Se pide:

1. Dibujar el grafo de operaciones correspondiente a cada función.
2. Utilizar el algoritmo de backpropagation para obtener el gradiente de f y g, evaluado en el punto indicado. Tip: recordar que el vector gradiente está formado por las derivadas parciales respecto a cada una de las variables que intervienen en la función. Indicar en el grafo de operaciones dibujado anteriormente los valores intermedios de las pasadas forward y backward.
3. Implementar dichas funciones en PyTorch y obtener el valor del gradiente utilizando las funcionalidades brindadas por la librería.

4. Cálculo de gradientes

Dada las siguientes definiciones:

I) a_i^l , es la salida de z_i^l luego de la función de activación.

$$a_i^l = \sigma(z_i^l)$$

II) z_j^l es lo computado por una unidad (neurona) j para la capa l , recibiendo todas sus entradas a_i^{l-1} . $w_{i,j}$ son los pesos de esa unidad, y b_j^l su sesgo.

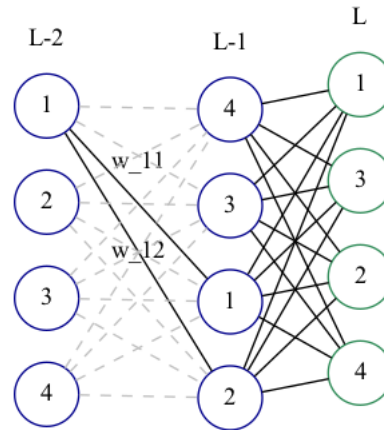
$$z_j^l = \sum_i w_{i,j} \cdot a_i^{l-1} + b_j^l$$

III) C es la función de costo, siendo L la última capa.

$$C = \sum_i \frac{1}{2} (y_i^L - a_i^L)^2$$

IV) $\frac{\partial C}{\partial z_j^L} = \delta_j^L$ es la derivada para la última capa.

V) $\frac{\partial C}{\partial z_j^l} = \delta_j^l$ es la derivada para cualquier capa.



Se pide demostrar las siguientes igualdades:

$$(1) \delta_j^L = (a_j^L - y_j^L) \cdot \sigma'(z_j^L)$$

$$(2) \delta_j^l = \sigma'(z_j^l) \sum_k \delta_k^{l+1} w_{jk}^{l+1} \text{ Sugerencia: considere usar la siguiente igualdad: } \delta_j^l = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l}, \text{ siendo } k \text{ el número de neuronas de la capa } l+1.$$

$$(3) \frac{\partial C}{\partial w_{ij}} = \delta_j^l \cdot a_i^{l-1}$$

$$(4) \frac{\partial C}{\partial b_j} = \delta_j^l$$

Conceptualmente, ¿qué representan los δ ?